

Projektskizze „Themenbrett“

Prof. Dr. Jens von Pilgrim

Zusammenfassung

Dieses Dokument gibt einen Überblick über die im Rahmen der Übungen zu erstellende Webapp.

Inhaltsverzeichnis

1 Vision und Systemkontext	2
2 UI-Skizzen	3
3 Use Cases	4
4 Domänenklassen	7
5 Architektur und Komponenten	8
6 Projekt-Plan	8
Literatur	9
Abkürzungsverzeichnis	9

1 Vision und Systemkontext

Stellen Sie sich vor, ein Kunde kommt zu Ihnen mit der Idee, eine Software für eine Webseite für die Vergabe von Abschlussarbeiten zu entwickeln. Er hat schon ein paar Ideen und Sie haben bereits in einem ersten Workshop mit ihm einige Dinge ausgearbeitet.

Im Rahmen des Studiums müssen Studenten am Ende eine Abschlussarbeit schreiben. Neben eigenen Themen werden häufig Themen von Professoren gestellt. Die Suche nach einem Thema kann relativ aufwendig sein, da es relativ viele Professoren gibt und den Studenten nicht immer klar ist, welcher Professor in welchem Gebiet Abschlussarbeiten vergibt.

Die Software soll eine Webanwendung namens “Themenbrett” sein, mittels der Professoren Themen ausschreiben können. Die Themen sollen in Gebiete eingeteilt werden, damit Studenten einfacher für sie interessante Themen finden können. Falls Studenten Interesse an einem Thema haben, aber das Thema noch nicht offiziell ausgegeben wurde, kann das Thema reserviert werden. Ausgegebene Themen können ebenfalls entsprechend gekennzeichnet werden.

Abbildung 1 zeigt das Systemkontextdiagramm. Die Anwendung ist sehr einfach, es sollen keine externen Systeme angebunden werden und es gibt nur zwei Akteure, nämlich Profs und Benutzer. Beide werden fast die gleiche GUI haben, nur dass Profs auch editieren können und ein paar mehr Felder sehen.

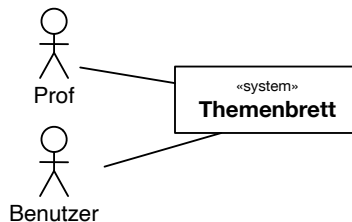


Abbildung 1: Systemkontextdiagramm



Systemkontextdiagramm und Notation

Systemkontextdiagramme sind ein erster Schritt in der Anforderungsermittlung. Die Kontextabgrenzung ist in [RP21, Kap. 2.2] erklärt, die Modellierung in [RP21, Kap. 3.4.2].

Sämtliche Diagramme verwenden Unified Modeling Language (UML). Zur UML gibt es neben der (mittlerweile lesbaren) Spezifikation [OMG17] sehr viele Einführungsbücher, hier müssen Sie selber einmal suchen. Ein Tipp generell: Halten Sie die Diagramme einfach und verwenden Sie nicht jedes Feature, das UML Ihnen bietet – weniger ist hier oft mehr.

Systemkontextdiagramme finden sich so übrigens nicht in [OMG17], obwohl die einzelnen Elemente durchaus UML-Elemente sind!

2 UI-Skizzen

Der Kunde hat sich auch schon Gedanken zum Graphical User Interface (GUI) gemacht. Das machen Kunden immer gerne, da wir das Frontend erst später entwickeln, lassen wir diese erst mal so stehen. Sie geben aber einen Eindruck davon, wie das System aussehen soll.

Zu Beginn sieht man einen Überblick mit den sichtbaren Gebieten (Abbildung 2). Wenn man auf ein Gebiet klickt, sieht man die einzelnen Themen mit den Betreuern (Abbildung 3). Wenn man auf ein Thema klickt, kann man dieses editieren (bzw. die Details ansehen, vgl. Abbildung 4). Wie üblich fehlt vieles, aber das soll uns erstmal nicht stören.



Abbildung 2: Übersicht aller Gebiete



Abbildung 3: Gebiet mit Themen



Abbildung 4: Details eines Themas

Vermutlich wird sich die GUI auch noch verändern, aber das werden wir später angehen.

3 Use Cases

Aufgrund der GUI-Skizzen und Gesprächen mit dem Kunden haben wir die in Abbildung 5 dargestellten Anwendungsfälle identifiziert.

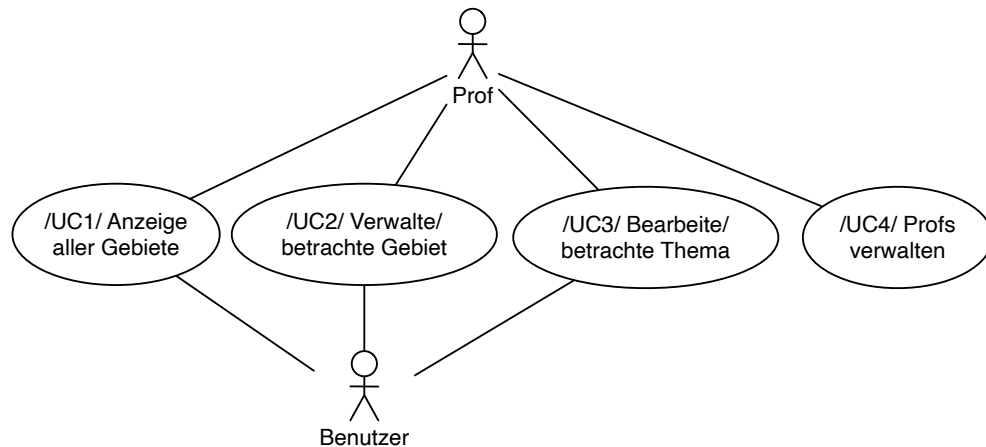


Abbildung 5: Anwendungsfälle

Die Use Cases sind im Folgenden einzeln beschrieben.

```

1  use case: /UC1/ Anzeige der Gebiete
2  actors: Benutzer bzw. Prof
3  main flow:
4      1. Gebiete werden mit Beschreibung und Anzahl der zugeordneten Themen angezeigt
5
6      Als Prof sollte man sehen, ob das Gebiet öffentlich ist oder bereits
7      geschlossen wurde und wer es wann erstellt hat.
8
9      Dabei werden nur Gebiete angezeigt, die Public sind oder bei denen der Prof
10     Ersteller ist.
11
12     2. Über die Anzeige soll der Benutzer bzw. Profs später zum Gebiet navigieren
13     können.
  
```

Listing 1: Use Case Anzeige der Gebiete

```

1  use case: /UC2/ Verwalte bzw. betrachte Gebiet
2  actors: Benutzer bzw. Prof
3  precondition: Ein Gebiet wurde ausgewählt oder ein neues soll erstellt werden, Prof
4  hat Zugriff auf das Gebiet
5  main flow:
6      1. Das Gebiet mit Beschreibung und Liste der Themen wird angezeigt. Als Prof
7      sieht man den Ersteller, das Datum der Erstellung sowie ob das Gebiet
8      öffentlich oder geschlossen ist.
9      Alle Themen des Gebiets werden mit Betreuer und Abschlussart angezeigt.
10  alternative flow: Bearbeite Gebiet (nur Profs)
11      1. Wenn der Prof Verwalter des Gebiets ist, kann sie oder er den Namen des
12      Gebiets editieren, das Gebiet veröffentlichen oder schließen -- dann können
13      keine neuen Themen mehr ergänzt werden --(bzw. beides rückgängig machen).
14      Dazu muss der Prof das Editieren einschalten.
15
16      2. Nach dem Ändern der Daten sind diese abzuspeichern. Falls die Daten doch
17      nicht geändert werden sollen, kann das Editieren abgebrochen werden
18
19      3. Das Gebiet wird angezeigt (Main Flow)
20  alternative flow: Neues Gebiet (nur Profs)
  
```

```

12      1. Wenn das Gebiet neu erstellt wird, muss der Name des Gebiets angegeben
13         werden.
13      2. Der Prof speichert dann das Gebiet, Änderungsdatum und Verwalter (der
14         erstellende Prof) werden automatisch eingetragen.
14      3. Die Liste wird angezeigt (Main Flow)
15      postcondition: Das Gebiet wurde angezeigt, Änderungen wurden gespeichert.
16      exceptional flow: Lösche Gebiet (nur Profs)
17      1. Wenn der Prof Verwalter des Gebiets ist, kann er das Gebiet löschen. Dabei
18         werden alle Themen des Gebiets ebenfalls gelöscht.
18      Achtung: Dabei werden auch Themen anderer Profs gelöscht, wenn diese dem Gebiet
19         zugeordnet waren.
19      2. Der Use Case wird dann verlassen.
20      postcondition: Gebiet und alle Themen wurden gelöscht
  
```

Listing 2: Use Case Verwalte Gebiet

```

1  use case: /UC3/ Bearbeite bzw. betrachte Thema
2  actors: Benutzer bzw. Prof
3  precondition: Prof ist angemeldet; ein Thema wurde ausgewählt oder soll erstellt
4                     werden; Benutzer hat Zugriff auf das Gebiet
4  main flow:
5      1. Das Thema wird angezeigt mit Titel, Beschreibung, Abschlussart, Betreuer, ob
6         das Thema offen oder reserviert ist sowie wann das Thema zuletzt
7         bearbeitet wurde.
6  alternative flow: Thema bearbeiten (nur Profs)
7      1. Wenn der Prof Ersteller des Themas ist, kann er Titel, Beschreibung,
8         Abschluss und Status editieren. Dazu muss der Prof das Editieren
9         einschalten.
8      2. Nach dem Ändern der Daten sind diese abzuspeichern. Falls die Daten doch
9         nicht geändert werden sollen, kann das Editieren abgebrochen werden
10     3. Das Thema wird angezeigt (Main Flow)
10  alternative flow: Neues Thema (nur Profs), nur möglich, wenn Gebiet nicht
11     geschlossen
11     1. Wenn ein Thema neu erstellt wird, müssen Titel, Beschreibung und
12        Abschlussart angegeben werden.
12     2. Der Prof speichert dann das Thema, Erstellungsdatum und Betreuer (der
13        erstellende Prof) werden automatisch eingetragen.
13  postcondition: Das Thema wird angezeigt, Änderungen wurden gespeichert.
14  exceptional flow: Lösche Thema
15     1. Wenn der Prof Ersteller des Themas ist, kann er das Thema löschen.
16     2. Der Use Case wird dann verlassen.
17  postcondition: Das Thema wurde gelöscht
  
```

Listing 3: Use Case Bearbeite Thema

```

1  use case: /UC4/ Profs verwalten
2  actors: Prof
3  precondition: Prof ist angemeldet und ist selbst Administrator (im folgenden Admin
4                     genannt)
4  main flow:
5      1. Alle Profs werden mit Titel, Name und CampusID (Login-Kennung) angezeigt.
6      2. Der Admin kann Titel, Name, CampusID, Passwort und Admin-Status ändern
7      3. Die Änderungen werden gespeichert
8  alternative flow: Neuen Prof erstellen (bei 1.)
9      1. Wenn der Prof neu erstellt wird, müssen Titel, Name, CampusID und initiales
10         Passwort angegeben werden.
10     2. Der Admin speichert den Prof. Alternativ kann er abbrechen, dann wird kein
11        neuer Prof angelegt.
11     3. Der Prof wird angezeigt (Main Flow)
  
```

```

12 alternative flow: Prof löschen
13   1. Der Admin löscht einen Prof. Er kann sich aber nicht selber löschen!
14   2. Auf Nachfrage werden alle Themen des Profs gelöscht.
15     % Falls der Prof Gebiete hat und diese keinen Themen anderer Profs enthält,
        werden diese gelöscht. Ansonsten wird der erste Prof mit einem Thema in
        dem Gebiet neuer Verwalter.
16     \textbf{Achtung: } Falls der Prof Gebiete verwaltet, werden auch diese
        gelöscht. Die in den Gebieten enthaltenen Themen werden ebenfalls
        gelöscht (auch, wenn sie von anderen Profs sind).
17   3. Die Liste der Profs wird aktualisiert und angezeigt (Main Flow)
18 postcondition: Die Profs wurden angezeigt, Änderungen wurden gespeichert.
  
```

Listing 4: Use Case Prof verwalten



Hinweise zu Use Cases

Zunächst ist es wichtig zu verstehen, dass das **Use-Case-Diagramm** (hier Abbildung 5) **nur ein Überblicksdiagramm** ist und im Grunde nicht viel aussagt. Tatsächlich sollte eine Anwendung in der Regel unter 10 Use Cases haben. Außerdem sollten Sie auf Beziehungen (includes, extends etc.) am besten verzichten.

Die eigentliche Information der Use Cases steckt woanders. Zum einen kann man **Use Cases textuell** beschreiben, was wir hier mit einfachen Templates gemacht haben. Diese Technik wird (leider nur sehr kurz) in [RP21, Kap. 3.3.2] besprochen. Die Use Cases enthalten bei uns drei 'Flows' (in deutsch 'Szenarien'): Einen **Main Flow** (Hauptszenario), der quasi den einfachsten Ablauf durch den Use Case beschreibt. Zusätzlich können **Alternative Flows** (Alternativ- oder Nebenszenarien) weitere, z.T. optionale Abläufe beschreiben. Im Fehlerfall (meist fachlich begründet) kann ein **Exceptional Flow** (Fehlerszenario) auftreten. Damit ein Use Case ausgeführt werden kann, müssen die **Preconditions** (Vorbedingungen) gelten. Diese sind extrem praktisch, da sie bspw. aufwändige Abfragen (ist der User eingeloggt etc.) ersparen. Alternative und Exceptional Flows sind sehr ähnlich. Der große Unterschied ist, dass Alternative und Main Flow dieselbe **Postcondition** haben. Exceptional Flows haben in der Regel eine eigene Postcondition. Alternativ oder ergänzend können Use Cases übrigens mit **Aktivitätsdiagrammen** beschrieben werden – diese geben den Ablauf (die Flows) graphisch an.

Sie werden evtl. auch schon von **User Stories** gehört haben, also die kurzen Beschreibungen von Aufgaben, die meist in der Form

"As a «role» I want to do «something» so that «some benefit»"

angegeben werden. Tatsächlich kann man Use Cases und User Stories zusammen verwenden: Dabei beschreiben mehrere User Stories einen Use Case; in der Regel wird pro Flow des Use Cases eine User Story erzeugt. Damit geben die Use Cases einen Kontext vor, in dem die User Stories eingebunden werden. So gesehen sind Use Cases eine Alternative zu Epics, die ebenfalls zur Gruppierung von User Stories eingesetzt werden.

4 Domänenklassen

Nach Analyse und Gesprächen mit dem Kunden wurde das Domänenklassendiagramm in Abbildung 6 erstellt.

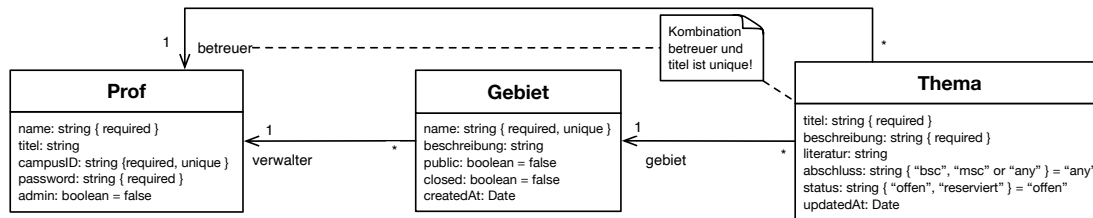


Abbildung 6: Domänenklassendiagramm

Ein Prof wird durch die CampusID identifiziert, aus dieser lässt sich auch die E-Mail ableiten ("campusID@bht-berlin.de"). Jeder Prof benötigt ein Passwort. Mit dem Admin-Flag kann man anzeigen, dass der Prof Administrator ist.

Ein Gebiet wird von einem Prof verwaltet. Das Gebiet enthält eine Beschreibung, ob es public (öffentlich) einsehbar ist, wann es erstellt wurde und ob es evtl. schon geschlossen wurde.

Ein Thema ist einem Gebiet zugeordnet und wird von einem Prof betreut. Beim Thema wird der Titel angegeben, eine Beschreibung, für welchen Abschluss das Thema geeignet ist und ob das Thema offen oder reserviert ist. Auch hat das Thema das Datum der letzten Änderung. Außerdem können Literaturhinweise angegeben werden.

Es ist zu beachten, dass der Betreuer eines Themas nicht notwendigerweise auch der Verwalter des dem Thema zugeordneten Gebiets sein muss!



Domänenklassendiagramm

Das Domänenklassendiagramm (oder Domänenmodell) wird manchmal auch als fachliches Klassendiagramm (oder Modell) bezeichnet, auf Englisch sagt man "domain (class) model". Das Entscheidende bei einem Domänen- oder fachlichen Klassenmodell ist, dass es strikt aus Sicht der Domäne, also des Anwendungsbereichs des Systems, erstellt wird. Im 'echten Leben' spricht man das Modell mit Fachexperten durch, also meist mit Nicht-Informatikern. Welchen Detailgrad man wählt, ist nicht strikt definiert. Auf jeden Fall sollte man auf Operationen und kompliziertere Strukturen (wie Assoziationsklassen, Unterscheidung von Interfaces oder Klassen, möglichst auch auf Vererbung) verzichten – das verstehen Fachexperten meist nicht. Man kann auch auf Typ-Angaben oder Multiplizitäten verzichten – wobei diese Angaben aus meiner Erfahrung sehr wichtig sind und auch von Anwendern verstanden werden. Hilfreich können abgeleitete (also berechnete) Attribute sein.

5 Architektur und Komponenten

Vom Kunden wurden bereits technische Randbedingungen genannt. So soll die Anwendung als Webapplikation umgesetzt werden. Zwischen Frontend und Backend soll eine REST-API verwendet werden, damit ggf. verschiedene Frontends (bspw. native Mobile-Apps) neben dem hier umzusetzenden Web-Frontend erstellt werden können. Die grundlegende Architektur ist im Komponentendiagramm in Abbildung 7 zu sehen.

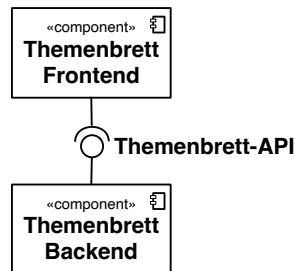


Abbildung 7: Client-Server-Architektur mit REST-API



Die Angabe von Komponentendiagrammen ist normalerweise eher Teil des Entwurfs. Im Kurs werden einige weitere Modelle bzw. Diagramme dafür verwendet. Mehr zum Thema Entwurf können Sie bspw. in [SKRG20] nachlesen. Wenn solche Dinge bereits wie hier im Rahmen von Anforderungsdokumenten angegeben werden, handelt es sich in der Regel um nicht-funktionale Anforderungen bzw. Randbedingungen.

6 Projekt-Plan

Das Projekt soll innerhalb von 16 Wochen umgesetzt werden. Dabei soll zunächst das Backend entwickelt werden, anschließend das Frontend. Es sind wöchentliche Abnahmen vereinbart. Dies entspricht den Übungsblättern und deren Lösungen.



Die Entwicklung von Projekten wird häufig durch sogenannte Vorgehensmodelle oder (Entwicklungs-) Prozesse definiert. Ein heutzutage häufig eingesetzter Prozess ist **Scrum** [SS20]. Dabei erstellt ein **Product Owner** ein Backlog, das alle zu erstellenden Aufgaben aufführt – häufig in Form von **User Stories** (siehe auch oben). Das **Team** (der Entwickler) arbeitet in sogenannten Sprints. Ein **Sprint** ist ein bis drei Wochen lang. Zu Beginn wählt das Team (in Absprache mit dem Product Owner) im Rahmen des **Sprint-Planning** Aufgaben aus dem Backlog aus und überführt diese in den **Sprint-Backlog**. Das Team verpflichtet sich, die Aufgaben des Sprint-Backlogs im nächsten Sprint umzusetzen! Daher darf während des Sprints dieses Sprint-Backlog auch nicht verändert werden. Am Ende des Sprints trifft sich das Team mit dem Product Owner und dem **Scrum Master** (der bei allen Treffen moderiert und die Ein-

haltung des Prozesses überwacht) und stellt seine Ergebnisse vor. Der Product Owner nimmt die Ergebnisse ab (oder auch nicht, dann wandern die Tasks wieder in das Backlog).

Betrachten Sie die Übung als einwöchige Sprints. Nun haben Sie keine Auswahl über die Tasks: Diese werden vom Betreuer (dem Product Owner) vorgegeben und damit unterscheidet sich natürlich eine Übung an der Hochschule wesentlich von einem echten Projekt. Tatsächlich können Sie aber die Aufgabenblätter als eine Art Sprint-Backlog ansehen, das Sie innerhalb meist einer Woche bearbeiten müssen. Da die Aufgaben aufeinander aufbauen, ist es wieder fast wie im echten Leben: Nicht erledigte Aufgaben müssen nachgeholt werden.

Literatur

- [OMG17] OMG: *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. formal/17-12-05. Needham, MA: Object Management Group, Dezember 2017. <https://www.omg.org/spec/UML/2.5.1/>
- [RP21] RUPP, Chris; POHL, Klaus: *Basiswissen Requirements Engineering*. 5. Auflage. dpunkt.verlag, 2021. ISBN 9783969102473
- [SKRG20] STARKE, Gernot; KOSCHEL, Arne; RAUSCH, Andreas; GHARBI, Mahbouba: *Basiswissen für Softwarearchitekten*. 4. Auflage. dpunkt.verlag, 2020. ISBN 9783969100127
- [SS20] SCHWABER, Ken; SUTHERLAND, Jeff: *Der Scrum Guide (Deutsche Ausgabe)*. Version: November 2020. <https://www.scrumguides.org/>. (CC BY-SA 4.0). – Online-Ressource

Abkürzungsverzeichnis

GUI	Graphical User Interface bzw. Benutzungsschnittstelle
UML	Unified Modeling Language [OMG17]